



UNIVERSIDADE FEDERAL DE SERGIPE
PRÓ-REITORIA DE PÓS-GRADUAÇÃO E PESQUISA
COORDENAÇÃO DE PESQUISA

PROGRAMA DE INICIAÇÃO CIENTÍFICA VOLUNTÁRIA – PICVOL

**Novas Estratégias para Otimização com Muitos
Objetivos e Suas Aplicações**

**Desenvolvimento de um Framework de Otimização
com Muitos Objetivos na Linguagem Python**

Relatório Final

Período da bolsa: de agosto de 2020 a julho de 2020

Este projeto é desenvolvido com bolsa de iniciação científica

PIBIC/COPES

SUMÁRIO

- 1. Introdução**
- 2. Objetivos**
- 3. Metodologia**
- 4. Resultados e discussões**
- 5. Conclusões**
- 6. Perspectivas de futuros trabalhos**
- 7. Referências bibliográficas**
- 8. Outras atividades**

1. Introdução

Problemas de Otimização Multiobjetivo (MOP - *Multi-Objective Optimization*, em inglês) são problemas na qual existem mais de uma função objetivo e essas precisam ser satisfeitas. Há sempre uma troca entre esses objetivos, pois eles estão sempre em conflito, ou seja, ao otimizar uma função objetivo, outra piora. Com isso, não existe apenas uma solução, mas sim um conjunto delas. Assim, é possível afirmar que a Otimização Multiobjetivo é definida pela obtenção de um conjunto de soluções que obedece à Otimalidade de Pareto (Coello, 2007), ou seja, um conjunto, denominado fronteira de Pareto, em que nenhuma solução domina outra, isto é, cada solução é melhor e pior, pelos critérios de avaliação de cada função objetivo, em, pelo menos, uma função objetivo, do que outra solução.

Dessa maneira, para a resolução dos MOPs, os algoritmos de otimização precisam encontrar esse conjunto das melhores soluções, considerando as várias soluções não dominadas. Nesse contexto, algoritmos evolucionários (Coello, 2007), dentre eles os algoritmos genéticos, apresentam bons resultados devido à sua característica populacional, a qual torna possível incorporar os conceitos da dominância de Pareto no processo de seleção das soluções e encontrar um conjunto de soluções em uma única execução. Na literatura, essa classe de algoritmos é conhecida como Algoritmos Evolucionários Multiobjetivo (MOEA, do inglês *Multi-objective Evolutionary Algorithm*).

Uma característica em comum dos MOEAs é a adoção de conceitos da evolução biológica. As soluções são os indivíduos da população. Inicialmente, os indivíduos dessa população cruzam entre si, dando origem a uma nova população. Em seguida, os indivíduos dessa nova população sofrem mutação. No fim, apenas os indivíduos mais aptos dessas duas populações são escolhidos, os quais passam a formar o conjunto das soluções. Todo esse processo se repete por um número definido de vezes.

O que diferencia um algoritmo baseado na evolução de outro é o modo como cada etapa é feita. Por exemplo, no cruzamento, todos ou alguns indivíduos podem participar. No caso de alguns indivíduos, estes podem ser os melhores, os piores ou ambos. Na mutação, a população inicial, ao invés da população gerada pelo cruzamento, pode ser a

afetada. Já, na etapa final, isto é, na seleção de indivíduos, podem ser selecionados os indivíduos mais distantes um do outro, os melhores indivíduos, metade dos melhores e dos piores indivíduos etc. É até possível que a ordem das etapas divirja da ordem apresentada, com a mutação antes do cruzamento, por exemplo.

Os algoritmos evolucionários têm sido aplicados com sucesso nos MOPs, mas eles apresentam dificuldades em problemas com 4 ou mais objetivos, também conhecidos como problemas com muitos objetivos. Nesses tipos de problemas, o número de soluções não dominadas cresce proporcionalmente ao número de objetivos (Deb e Jain, 2014), o que lesa o aspecto exploratório, ou seja, as soluções ficam concentradas em um espaço, carecendo, assim, a pressão em direção à fronteira de Pareto. Tanto que novos algoritmos menos propensos a esse problema vêm sendo propostos, como o NSGA-III (Deb e Jain, 2014).

Contudo, um problema se mantém presente. As funções objetivas são computacionalmente exigentes, tanto que, em algumas situações, torna a otimização multiobjetiva inviável (Deb et al., 2019). Com isso, foram propostos algoritmos de otimização multiobjetiva que fazem o uso de surrogates, trocando precisão por tempo. Tais algoritmos estão cada vez mais populares, pois se mostraram eficazes na otimização multiobjetiva e em questão computacional.

Como visto, existem vários algoritmos multiobjetivo propostos, cada um com seu propósito. No entanto, o código fonte desses algoritmos não é disponibilizado ou utiliza uma linguagem de programação diferente. Isso dificulta a reprodução dos experimentos e o uso desses algoritmos em novas comparações. Desse modo, para evitar esses problemas, novos frameworks buscam juntar algoritmos multiobjetivos da literatura e ferramentas, como métricas de qualidade e alguns problemas de benchmarking, que auxiliam no desenvolvimento de novos algoritmos.

Apesar da existência desses frameworks, sendo jMetal, DeepMaker e METCO alguns exemplos, poucos utilizam a linguagem Python e incorporam novas técnicas, como os algoritmos de surrogate.

2. Objetivos

O objetivo deste trabalho é desenvolver um framework em Python com a implementação dos principais algoritmos MOPs da literatura. Busca-se que o framework seja extensível, ao passo que incorpora a implementação de problemas de benchmark e de métricas de qualidade. Também são feitos experimentos com o propósito de validar os algoritmos propostos.

3. Metodologia

O desenvolvimento do framework foi dividido em duas partes. A primeira, sob responsabilidade deste plano de trabalho, buscou o projeto e a construção do framework. Já a segunda buscou o desenvolvimento de algoritmos baseados em surrogates, os quais foram incorporados ao framework.

Foram implementados três dentre os principais algoritmos na literatura: NSGA-II (Deb et al., 2002), NSGA-III (Deb e Jain, 2013) e MOEA/D (Zhang e Li, 2007).

Também foram estudados outros frameworks de algoritmos multiobjetivos já validados, para a compreensão da estrutura dos mesmos.

Nota-se que, no mês de junho, o aluno que estava desenvolvendo este plano de trabalho pediu desligamento. Então, o plano foi assumido pelo aluno que estava desenvolvendo os algoritmos baseados em surrogates, ou seja, o aluno responsável pela segunda parte do desenvolvimento do framework. Assim, a parte final do plano se concentrou na escrita do relatório, na escrita de um artigo relatando os resultados dos algoritmos baseados em surrogate e na integralização dos códigos num único repositório, incorporando os algoritmos desenvolvidos em ambos os planos de trabalho.

3.1 Fundamentação teórica

O Problema de Otimização Multi-Objetivo (MOP - *Multi-Objective Optimization Problem*, em inglês) é definido com a minimização ou maximização de uma função $F(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_k(\mathbf{x}))$ sujeito à $g_i(\mathbf{x}) \leq 0$, $i = \{1, \dots, m\}$ e $h_j(\mathbf{x}) = 0$, $j = \{1, \dots, p\}$, onde $\mathbf{x}$ é um vetor de variáveis de decisão $\mathbf{x} = (x_1, \dots, x_n)$ de algum universo Ω . $g_i(\mathbf{x}) \leq 0$ e $h_j(\mathbf{x}) = 0$ representam restrições que precisam ser respeitadas enquanto minimiza (ou maximiza)

$F(\mathbf{x})$ (Coello *et al.*, 2006).

Para solucionar os MOPs, alguns algoritmos utilizam do conceito de Fronteira de Pareto que é definida a partir de dois outros conceitos:

- Dominância de Pareto: dado um vetor $\mathbf{u} = (u_1, \dots, u_k)$ é dita que este domina outro vetor $\mathbf{v} = (v_1, \dots, v_k)$ (denotado por $\mathbf{u} \prec \mathbf{v}$) se, e somente se, $\forall i \in \{1, \dots, k\}, u_i \leq v_i \wedge \exists i \in \{1, \dots, k\} : u_i < v_i$.
- Conjunto Ótimo de Pareto: $P^{\ell} := \{\mathbf{x} \in \Omega \mid \neg \exists \mathbf{x}' \in \Omega \ F(\mathbf{x}') \prec F(\mathbf{x})\}$.

Assim sendo, a Fronteira de Pareto é definida como: $PF^{\ell} := \{\mathbf{u} = F(\mathbf{x}) \mid \mathbf{x} \in P^{\ell}\}$.

Visando a solução desses problemas, foram implementados os seguintes algoritmos: NSGA-II e NSGA-III.

O NSGA-II (Deb *et al.*, 2002) é um algoritmo evolucionário que se destaca pela seleção de soluções via ordenação a priori com os algoritmos, os quais foram propostos no mesmo artigo, fast non dominated sorting e crowding distance.

O fast non dominated sorting cria, baseado em duas regras, grupos. A primeira regra é que as soluções de um mesmo grupo não se dominam, ou seja, nenhuma é melhor ou pior do que outra, e a segunda regra é que as soluções de um grupo dominam todas as soluções de grupos inferiores. Por exemplo, se há 6 grupos, o grupo 1 domina todos os outros 5 grupos, o 2 domina os outros 4, o 3 domina os grupos 4, 5 e 6.

Já o outro algoritmo, o crowding distance, ordena decrescentemente as soluções com base na distância do valor da função objetivo, isto é, as soluções com os valores da função objetivo discrepantes primeiro. Desse modo, a ordenação do NSGA-II se dá selecionando as soluções dos grupos dominantes até que não haja mais grupos ou a adição de um grupo faça com que o tamanho da população selecionada ultrapasse um limite de tamanho populacional imposto. Quando isso acontecer, são selecionadas as primeiras soluções do grupo que iria ser selecionado após ser ordenado pelo crowding distance.

O NSGA-III (Deb e Jain, 2013), é um algoritmo evolucionário baseado em pontos de referência. Em suma, ele é semelhante ao NSGA-II, com exceção do crowding

distance. No lugar desse algoritmo, é feita a seleção da população restante baseada em agrupamentos. A premissa desses agrupamentos é que há um grupo para cada direção de referência e a uma solução é designado o grupo da direção de referência mais próxima à solução. Então, para cada grupo, são selecionadas as soluções com a menor distância. Se o tamanho da população ainda não for preenchido, são selecionadas soluções aleatórias de cada grupo.

Também foram implementados os seguintes algoritmos com surrogate: M1, M3, M6, SMOyO e SMByO.

Os algoritmos M1, M3 e M6 (Deb et al., 2019), inicialmente, geram uma população inicial aleatória. A cada passo, eles executam algum algoritmo evolucionário por um determinado número de gerações, adicionando a fronteira de Pareto encontrada àquela população inicial. É importante observar que o cálculo dos objetivos no algoritmo evolucionário é feito utilizando o surrogate. O treinamento desse surrogate se dá a cada τ passos, além de ser feito com a função objetivo e com a população inicial. Quando o número de avaliações com a função objetivo for maior ou igual a SE_{max} , o algoritmo para.

O M3 e M6 transformam, a priori, o problema multiobjetivo em um problema com um único objetivo. O M3 utiliza a função ASF (Wierzbicki, 1980) para essa transformação. Já o M6, a função KKTPM (Deb e Abouhawwash, 2016). Para o uso dessas funções, são necessárias direções de referência, as quais devem ter as mesmas dimensões da função multiobjetivo.

Os algoritmos SMOyO e SMByO (Oliveira, 2020) são semelhantes. Entretanto, nem sempre executam o algoritmo evolucionário com o surrogate. O uso do surrogate só ocorre após t_{max} avaliações com a função objetivo. Além disso, o algoritmo para quando atinge t_{parada} avaliações com a função objetivo e com o surrogate. Uma outra diferença é que a população utilizada nos treinos é formada pela fronteira de Pareto do algoritmo evolucionário e os indivíduos dominantes da população atual.

O SMOyO difere do SMByO no fato de que o treinamento é feito parcialmente, isto é, enquanto o critério t_{max} não é atingido, a cada iteração, utiliza a população atual

para treinar o surrogate. Por conseguinte, o SMByO só treina o surrogate quando tmax é atingido.

3.1. Frameworks

Objetivando a compreensão do desenvolvimento do Framework foi feita uma revisão bibliográfica levando em consideração as seguintes informações: linguagem de programação, tipo de solução, algoritmos, reusabilidade, indicadores de qualidade e problemas de *benchmark*. A seguir encontram-se os frameworks selecionados para a revisão:

- O SIMEON (Halim *et al.*, 2011) é um framework feito em Java que aceita soluções do tipo binário, real, inteiro e inteiro-real. E possui uma implementação do algoritmo NSGA-II.
- METCO (Leon *et al.*, 2009) é um framework extensível feito em C++ que possui implementações dos algoritmos: SPEA (Zitzler, Eckart, e Lothar Thiele, 1998), SPEA2 (Eckart *et al.*, 2001), NSGA (Srinivas *et al.*, 1994), NSGA-II e IBEA (Zitzler, Eckart e Simon Künzli, 2004). Esse framework também dispõe dos indicadores de qualidade: contribution, hypervolume, binary multiplicative epsilon indicator. E implementa problemas de benchmark da família ZDT.
- JMetal (Durillo *et al.*, 2011) é um framework desenvolvido em Java que possui soluções genéricas e implementações dos algoritmos: NSGA-II, SPEA2, PAES (Knowles *et al.*, 2000), PESA-II (Corne *et al.*, 2001), OMOPSO (Sierra *et al.*, 2005), MOCcell (Nebro *et al.*, 2007), AbYSS (Nebro *et al.*, 2008), MOEA/D, Denssea (Greiner *et al.*, 2007), CellDE (Durillo *et al.*, 2008), GDE3 (Kukkonen, Saku, e Jouni Lampinen, 2005), FastPGA (Eskandari *et al.*, 2007), IBEA, SMPSO (Nebro *et al.*, 2009), MOCHC (Nebro *et al.*, 2007), SMS-EMOA (Beume *et al.*, 2007). Esse framework possui os indicadores de qualidade: Epsilon Indicator, Spread, Hypervolume. E nele foram implementados os problemas de benchmark: ZDT, DTLZ, WFG, CEC2009, Li-Zhang.
- VIDEO (Kollat *et al.*, 2007) é um framework desenvolvido em Python com implementação do algoritmo Epsilon-NSGAIL.
- ParadisEO-MOEO (Liefvooghe *et al.*, 2007) é um framework feito em C++ que

possui soluções do tipo real, nele foram implementados os algoritmos: MOGA, NSGA, NSGA-II, SPEA, SPEA2, IBEA. Esse framework é expansível por meio de herança ou especialização e possui os indicadores de qualidade: entropy e contribution.

- JCLEC (Ventura *et al.*, 2008), feito em Java, é um framework que possui soluções genéricas e nele são implementados os algoritmos: NSGA-II e SPEA2.
- Jeaf (Caamaño *et al.*, 2010) é uma framework com soluções genéricas desenvolvido em Java que possui implementações de algoritmos do tipo: Genetic Algorithms, Evolutionary Strategies, Differential Evolution, CMA-ES, Micro-genetic Algorithms, Macroevolutionary Algorithms. Esse framework possui os problemas de benchmark: CEC 2005-2007, Dixon-Szegö function set.
- Open BEAGLE (Gagné, Christian, e Parizeau, 2006), desenvolvido em C++, esse framework possui uma implementação do NSGA-II.

3.2. Desenvolvimento

O Framework desenvolvido leva em consideração apenas soluções do tipo real. Um estudo foi feito no início do seu desenvolvimento sobre Padrões de Projetos e a criação do Diagrama de Classes, mostrado na Figura 1. O padrão de projeto *Singleton* foi selecionado para a implementação da classe que representa a Fronteira de Pareto.

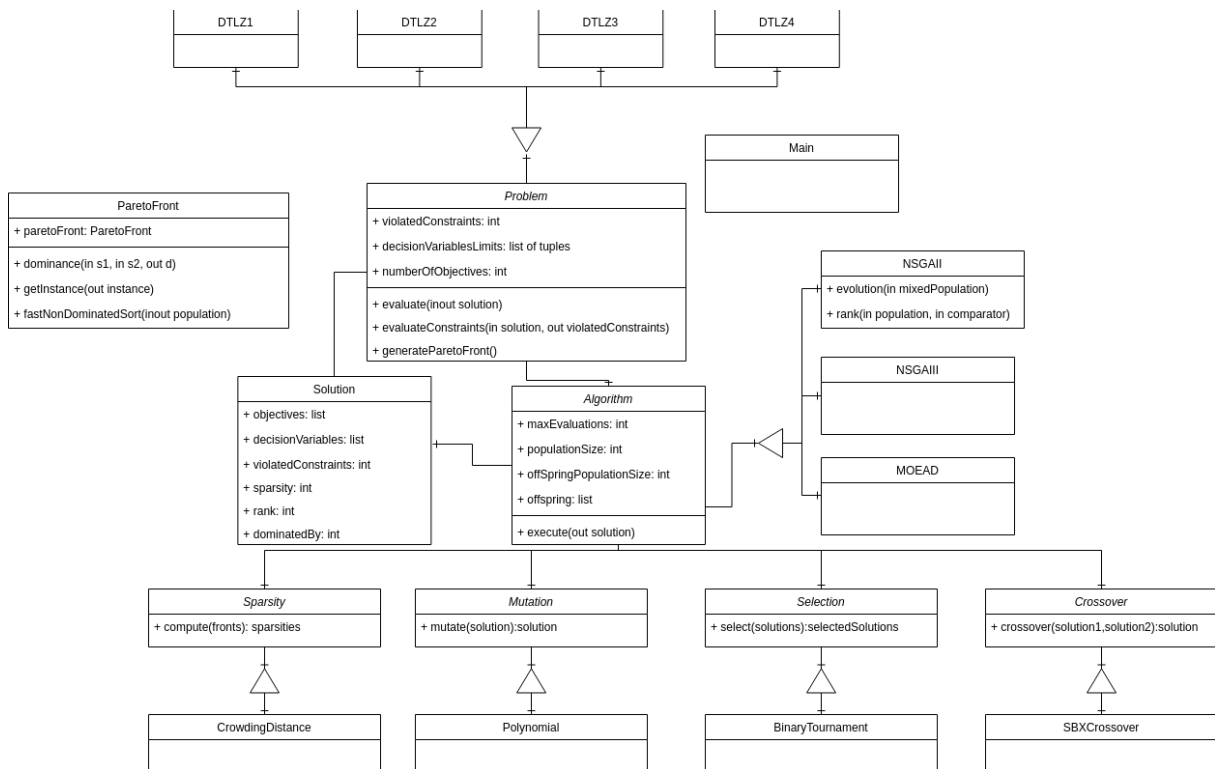


Figura 1: Diagrama de Classes do Framework.

No Diagrama de Classes é possível ver as seguintes classes:

- *Problem*: classe abstrata que representa os problemas que serão otimizados.
- *Solution*: classe que implementa soluções do tipo real, nela encontram-se as variáveis de decisão e o vetor função.
- *ParetoFront*: classe que contém a Fronteira de Pareto e os principais métodos relacionados a esta fronteira.
- *Algorithm*: classe abstrata que representa os algoritmos que serão implementados. Esta contém uma referência aos operadores evolucionários.
- *Sparsity*: classe abstrata que representa a medida da população.
- *Mutation*: classe abstrata que representa a operação de mutação.
- *Selection*: classe abstrata que representa a operação de seleção.
- *Crossover*: classe abstrata que representa a operação de recombinação.
- *Main*: é a classe principal de entrada e saída do framework. Nela serão definidos quais algoritmos serão utilizados, seus parâmetros e qual o problema que será otimizado.

3.2.1. Algoritmos Desenvolvidos

O código do framework se encontra no repositório <https://github.com/DCOMP-UFS/PyManyobjective>. Nele, foram implementados os algoritmos NSGA-II e NSGA-III da classe Algorithm. Da classe Sparsity, foi implementado o crowding distance, um dos algoritmos de seleção utilizado pelo NSGA-II. Já das classes mutation e crossover foram implementados o Polynomial Mutation e o SBX Crossover (Deb, Sindhya e Okabe, 2007).

O SBX Crossover combina os valores de cada variável de duas soluções. A depender do índice de distribuição, o resultado será próximo ou distante das soluções usadas. O algoritmo de mutação Polynomial Mutation segue este mesmo princípio de distribuição.

De Selection também houve apenas um, Binary Tournament, o qual seleciona, entre duas soluções, a dominante. De Problem, foram implementados todos os problemas utilizados nos testes dos algoritmos evolucionários e dos algoritmos com surrogates, ou seja, os problemas ZDT1, ZDT3, ZDT6, DTLZ1, DTLZ2, DTLZ3, DTLZ4, DTLZ5, DTLZ6 e DTLZ7.

A medida de qualidade implementada foi o IGD e os algoritmos com surrogates implementados foram M1, M3, M6, SMOyO e SMByO.

4. Resultados e discussões

Com o propósito de validar a implementação dos algoritmos, foi feita uma avaliação em problemas de benchmark, cujos resultados foram comparados com os valores das métricas de qualidade encontrados em artigos da literatura e com o desempenho dos mesmos algoritmos em outros frameworks.

Os resultados foram divididos em dois conjuntos. O primeiro conjunto contém os experimentos com os algoritmos básicos, isto é, os algoritmos evolucionários, NSGA-II e NSGA-III. Enquanto que o segundo conjunto contém os experimentos com os algoritmos de surrogate M1, M3, M6, SMOyO e SMByO.

4.1. Problemas

Para a realização dos experimentos do NSGA-II e NSGA-III, foram considerados os problemas de Benchmark da família DTLZ (Deb *et al.*, 2002). Já, para a realização dos experimentos com os algoritmos M1, M3, M6, SMOyO e SMByO, foram considerados alguns problemas da família ZDT (Deb, Zitzler e Thiele, 2000), isto é, os problemas ZDT1, ZDT3 e ZDT6, e todos da família DTLZ.

Os problemas da família ZDT têm o propósito de avaliar a convergência à fronteira de Pareto e a manutenibilidade da diversidade populacional de um algoritmo multiobjetivo. Com relação à convergência, são identificados quatro aspectos: multimodalidade, decepção, ótimos isolados e ruído colateral. Já em relação à manutenibilidade, são: convexidade, discrição e não uniformidade (Deb, 1999).

Em suma, os problemas da família ZDT avaliam esses aspectos. Dos problemas escolhidos, tem-se que o problema ZDT1 avalia a convexidade, ou seja, a capacidade de favorecer soluções intermediárias. Já o ZDT3, a discrição, a capacidade de encontrar soluções em regiões não contínuas. Enquanto o ZDT6, a não uniformidade, isto é, a capacidade de encontrar soluções em regiões menos densas.

A família DTLZ tem uma proposta semelhante à família ZDT. Cada problema, do 1 ao 7, testa, respectivamente, convergência à fronteira de Pareto, escalabilidade da performance em problemas com muitos objetivos, convergência com muitos ótimos locais, manutenibilidade da diversidade populacional, a convergência a uma curva degenerada, manutenibilidade de soluções em diferentes regiões e convergência a regiões com diferentes formas.

4.2. Algoritmos e parâmetros

Os experimentos dos algoritmos básicos foram feitos em cima de três e oito objetivos para cada problema utilizando o NSGA-II e NSGA-III. No NSGA-III para três objetivos foi utilizado 12 divisões e para oito objetivos, 8 divisões. No problema DTLZ1, foi utilizado $k=5$, e dos outros problemas, $k=10$, onde k define a quantidade de variáveis de decisão de acordo com a expressão: $n=k+m-1$, m sendo a quantidade de objetivos.

Nos experimentos dos algoritmos com surrogates, para os problemas da família ZDT, usou-se indivíduos com 10 variáveis e, para os da família DTLZ, com 7 variáveis.

O parâmetro k dos DTLZs foi 5.

Nos algoritmos M1, M3 e M6, o surrogate utilizado foi o Kriging (Jones, 2001) e os parâmetros utilizados foram 500 para SE_{max} , 300 para o tamanho da população inicial e 50 para τ , lembrando que SE_{max} é o número máximo de avaliações com a função objetivo e τ é o número que define quando os surrogates serão treinados. Precisamente, o surrogate é treinado quando o valor do passo atual é um múltiplo de τ .

Nos algoritmos SMOyO e SMByO, o surrogate utilizado foi o Random Forest (Breiman, 2001) e os parâmetros utilizados foram 100 para o tamanho da população inicial, 500 para t_{max} e 10000 para t_{parada} . Observa-se que t_{max} é o número máximo de avaliações com a função objetivo e t_{parada} é o valor do produto entre o número de execuções do framework e o tamanho da população do algoritmo evolucionário.

Nota-se que os parâmetros dos algoritmos M1, M3 e M6 foram baseados nos parâmetros utilizados no artigo de origem. Já os parâmetros dos algoritmos SMOyO e SMByO, foram escolhidos de forma que o número de avaliações com a função objetivo, com os surrogates e com o algoritmo evolucionário seja mais próximo possível do número dos algoritmos M1, M3 e M6.

Para todos os experimentos, temos: $p_R=1$ e $p_M=1/n$, que são as probabilidades de recombinação e mutação, respectivamente. Para o NSGA-II, foi utilizada uma população com 100 indivíduos.

4.3. Medidas de qualidade

Como medida de qualidade, foi selecionada a distância inversa generacional (IGD - *Inverse Generational Distance*, em inglês) (Coello e Sierra, 2004), a qual proporciona várias informações em relação à diversidade e convergência das soluções, e também, é a métrica utilizada em (Deb e Jain, 2014), que são os valores de referência escolhidos para validar os algoritmos aqui implementados.

4.4. Resultados dos algoritmos básicos

Nessa seção é apresentado os resultados do experimento com os algoritmos NSGA-II e NSGA-III, para 3 e 8 objetivos. Foram feitas 20 execuções para cada

experimento, obtendo-se os resultados para os problemas: DTLZ1 (Tabela 1), DTLZ2 (Tabela 2), DTLZ3 (Tabela 3), e DTLZ4 (Tabela 4).

	NSGA-II				NSGA-III	
Valores	M	MaxGen	IGD	IGD (Durillo et al., 2011)	IGD	IGD (Deb e Jain, 2014)
Melhor	3	400	2,66E-01	3,39E-01	1,90E-01	4,88E-04
Médio			3,39E-01	7,41E-01	3,28E-01	1,31E-03
Pior			4,66E-01	1,16E+00	3,92E-01	4,88E-03
Melhor	8	750	1,00E+00	5,32E+00	4,34E-01	2,04E-03
Médio			1,28E+00	1,08E+01	9,12E-01	3,98E-03
Pior			1,68E+00	2,09E+01	1,42E+00	8,72E-03

Tabela 1: Resultados para o DTZ1.

	NSGA-II				NSGA-III	
Valores	M	MaxGen	IGD	IGD (Durillo et al., 2011)	IGD	IGD (Deb e Jain, 2014)
Melhor	3	250	4,23E-03	3,47E-03	4,34E-03	4,88E-04
Médio			4,99E-03	3,98E-03	5,16E-03	1,31E-03
Pior			5,63E-03	4,60E-03	6,27E-03	4,88E-03
Melhor	8	500	5,41E-02	3,87E-02	3,07E-02	2,04E-03
Médio			5,78E-02	4,47E-02	3,16E-02	3,98E-03
Pior			6,47E-02	5,02E-02	3,35E-02	8,72E-03

Tabela 2: Resultados para o DTLZ2.

	NSGA-II				NSGA-III	
Valores	M	MaxGen	IGD	IGD (Durillo et al., 2011)	IGD	IGD (Deb e Jain, 2014)
Melhor	3	1000	2,81E+00	1,71E+00	2,63E+00	4,88E-04
Médio			3,25E+00	3,36E+00	3,19E+00	1,31E-03
Pior			3,56E+00	5,22E+00	3,53E+00	4,88E-03
Melhor	8	1250	1,09E+01	1,93E+01	8,51E+00	2,04E-03
Médio			1,31E+01	3,21E+01	1,53E+01	3,98E-03
Pior			1,54E+01	4,13E+01	1,84E+01	8,72E-03

Tabela 3: Resultados para o DTLZ3.

	NSGA-II				NSGA-III	
Valores	M	MaxGen	IGD	IGD (Durillo et al., 2011)	IGD	IGD (Deb e Jain, 2014)
Melhor	3	600	6,84E-03	3,90E-03	7,03E-03	4,88E-04
Médio			8,30E-03	6,97E-03	8,07E-03	1,31E-03
Pior			9,45E-03	9,68E-03	9,13E-03	4,88E-03
Melhor	8	1250	7,09E-02	4,34E-02	4,56E-02	2,04E-03
Médio			7,64E-02	5,01E-02	4,69E-02	3,98E-03
Pior			8,32E-02	5,31E-02	4,90E-02	8,72E-03

Tabela 4: Resultados para o DTLZ4.

Como referência de valores do IGD para o NSGA-II e NSGA-III, foram utilizados o *framework* JMetal (Durillo *et al.*, 2011) e os experimentos presentes em (Deb e Jain, 2014), respectivamente.

A partir dos resultados, é possível observar uma diferença mínima entre o NSGA-II implementado aqui e o outro implementado em (Durillo *et al.*, 2011), em alguns momento o IGD aparece menor no framework proposto neste trabalho. Já o NSGA-III tem resultados bastante próximos dos vistos em (Deb e Jain, 2014), com exceção da Tabela 3, onde esses divergem.

4.5. Resultados dos algoritmos baseados em surrogate

Nesta seção, são apresentados os resultados do experimento com os algoritmos M1, M3, M6, SMOyO e SBMyO. Foram feitas 30 execuções para cada problema. Os resultados são disponibilizados a seguir.

	M1		M3		M6		SMOyO		SMBByO	
	Média	SD	Média	SD	Média	SD	Média	SD	Média	SD
ZDT1	0.0081	0.0012	0.0471	0.0165	0.0392	0.0649	0.0757	0.0624	0.0275	0.0068
ZDT3	0.0147	0.0129	0.3084	0.0876	0.5112	0.0976	0.1127	0.0542	0.2948	0.0081
ZDT6	2.4293	0.8372	0.6128	0.165	5.7185	0.8776	2.3215	1.2989	0.6961	0.1303
DTLZ1	35.6289	11.3859	6.0579	2.921	33.2907	9.067	0.2941	0.1511	7.6727	13.2727
DTLZ2	0.0777	0.0053	0.0850	0.0024	0.0554	0.0061	0.3498	0.1308	0.7025	0.0339
DTLZ3	87.8393	24.5932	21.9781	8.874	97.1692	29.6277	2.7725	2.6142	26.8281	34.0127
DTLZ4	0.1954	0.0375	0.1463	0.0407	0.2238	0.0268	0.2489	0.0771	0.2984	0.0033
DTLZ5	0.0110	0.0033	0.0232	0.0025	0.0132	0.0016	0.1268	0.0555	0.0625	0.03
DTLZ6	3.2437	0.2102	2.2143	0.1644	3.8764	0.1432	3.8276	0.5288	1.0233	0.2475
DTLZ7	0.0925	0.0108	0.8309	0.3493	1.8749	1.1868	1.7805	1.4669	0.3025	0.162

Nessa tabela, são dispostos a média e o desvio padrão dos 30 IGDs dos algoritmos M1, M3, M6, SMOyO e SMBByO, respectivamente, da esquerda para a direita. Sendo assim, cada linha da tabela representa esses dados para um dos problemas escolhidos. Nesse caso, os problemas ZDT1, ZDT3, ZDT6, DTLZ1, DTLZ2, DTLZ3, DTLZ4, DTLZ5, DTLZ6 e DTLZ7, respectivamente, de cima para baixo.

Vale salientar que todos esses dados foram obtidos ao se executar os algoritmos nos problemas utilizando os parâmetros mencionados anteriormente.

Com relação ao desempenho, no problema ZDT1, o algoritmo M1 foi o que apresentou a menor média e o que se mostrou mais consistente. Depois dele, em ordem crescente de média, vêm os algoritmos SMBByO, M6, M3 e SMOyO, respectivamente. O M6, em relação ao M3, apesar de ter uma média menor, possui um desvio padrão maior.

No problema ZDT3, o algoritmo M1 também foi o melhor. Ele foi seguido dos algoritmos SMOyO, SMBByO, M3 e M6, respectivamente. O SMBByO teve média menor do que o SMOyO, mas maior desvio padrão. O M3 ganha, em todos os aspectos, do M6.

No ZDT6, quem obteve a menor média, dessa vez, foi o M3. O SMBByO obteve resultados semelhantes, apresentando média maior e desvio padrão menor. A classificação segue com os algoritmos SMOyO, M1 e M6, respectivamente. O SMOyO, apesar de média menor do que o M1, teve maior desvio padrão.

O DTLZ1 favoreceu o SMOyO, o qual teve menor média e desvio padrão. A

classificação segue com M3, SMByO, M1 e M6, respectivamente. O SMByO, no entanto, teve o maior desvio padrão.

No DTLZ2, o M6 foi o campeão, obtendo a menor média. Em seguida, seguem o M1 e o M3, respectivamente. A média e o desvio padrão desses três algoritmos foi semelhante. Vale notar que ambos o M1 e o M3 obtiveram desvio padrão menor do que o M6. Atrás, seguem o SMByO e o SMOyO. Apesar do SMByO ter menor média, teve maior desvio padrão, em relação ao SMOyO.

No DTLZ3, o SMOyO venceu, tanto em média quanto em desvio padrão. Em seguida, vêm M3, SMByO, M6 e M1, respectivamente. O SMByO, apesar de ter média menor do que os algoritmos M6 e M1, teve maior desvio padrão.

No DTLZ4, o M3 obteve a menor média. Ele foi seguido pelos M1, M6, SMOyO e SMByO, respectivamente. SMByO, apesar de ser o pior em média, teve o menor desvio padrão. Tanto o M1 quanto o M6, também obtiveram um desvio padrão menor do que o M3. Assim, o M3 só teve desvio padrão menor do que o SMOyO.

Já no DTLZ5, todos os algoritmos desempenharam bem. No entanto, quem conseguiu a menor média foi o M1, seguido pelos algoritmos M6, M3, SMByO e SMOyO, respectivamente. O M6 teve o menor desvio padrão. Depois, o M3. Só então, chega o M1.

O SMByO teve a menor média no DTLZ6. Depois, vem o M3, o qual conquistou um desvio padrão menor. Em sequência, tem-se M1, SMOyO e M6, respectivamente. O M6, apesar de maior média, teve um desvio padrão menor do que o M1 e o SMOyO.

No DTLZ7, o M1 teve o menor médio e o menor desvio padrão. Em seguida, em ordem crescente de médio e desvio padrão, vêm os algoritmos SMByO, M3, SMOyO e M6.

Ao se comparar os resultados das implementações com os resultados dos artigos originais, tem-se que, no caso dos frameworks do Deb, todos, com exceção do M3, foram semelhantes, tanto em média quanto em desvio padrão, e, no caso dos frameworks de Joel, as implementações obtiveram um resultado cerca de dez vezes pior.

5. Conclusões

Focando-se em projetar e implementar um framework em Python extensível, foram implementados 2 algoritmos evolucionários da literatura e 7 algoritmos baseados em surrogates. Além disso, foram realizados testes para a validação das implementações.

O framework se encontra disponível no GitHub para extensões em novos trabalhos. Vale ressaltar que, diferente dos demais frameworks, este disponibiliza algoritmos com surrogate, os quais podem ser aplicados a novos problemas incorporados no framework.

Os algoritmos evolucionários implementados foram NSGA-II e NSGA-III. Já os frameworks, M1, M3, M6, SMOyO e SMByO. Pelos resultados, conclui-se que a implementação de 3 dos 6 algoritmos baseados em surrogate é capaz de obter resultados semelhantes aos apresentados no estado da arte. O mesmo é dito dos algoritmos evolucionários, os quais apresentaram valores de IGD satisfatórios.

6. Perspectivas de futuros trabalhos

Para futuros trabalhos, há espaço para a incorporação de novos algoritmos, como o MOEA/D, e para a expansão dos experimentos com outros problemas de benchmarking. Além disso, resta comparar os algoritmos baseados em surrogate com os algoritmos base.

7. Referências bibliográficas

BEUME, Nicola, NAUJOKS, Boris e EMMERICH, Michael. "SMS-EMOA: Multiobjective selection based on dominated hypervolume." *European Journal of Operational Research* 181.3 (2007): 1653-1669.

CAAMANÕ, Pilar et al. "Jeaf: A java evolutionary algorithm framework. *IEEE Congress on Evolutionary Computation.*" IEEE, 2010.

COELLO, Carlos A. Coello, LAMONT, Gary B. e VAN VELDHUIZEN, David A. "Evolutionary algorithms for solving multi-objective problems." Vol. 5. New York: Springer, 2007.

Corne, David W., et al. "PESA-II: Region-based selection in evolutionary multiobjective optimization." Proceedings of the 3rd annual conference on genetic and evolutionary computation. 2001.

DEB, Kalyanmoy, et al. "A fast and elitist multiobjective genetic algorithm: NSGA-II." IEEE transactions on evolutionary computation 6.2 (2002): 182-197.

DEB, Kalyanmoy, et al. "Scalable multi-objective optimization test problems." Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No. 02TH8600). Vol. 1. IEEE, 2002.

DURILLO, Juan J., e NEBRO, Antonio J. "jMetal: A Java framework for multi-objective optimization." Advances in Engineering Software 42.10 (2011): 760-771.

DURILLO, Juan J., et al. "Solving three-objective optimization problems using a new hybrid cellular genetic algorithm." International Conference on Parallel Problem Solving from Nature. Springer, Berlin, Heidelberg, 2008.

ECKART, Z., MARCO, L. e LOTHAR, T. "Improving the strength Pareto evolutionary algorithm for multiobjective optimization." EUROGEN, Evol. Method Des. Optim. Control Ind. Problem (2001): 1-21.

ESKANDARI, Hamidreza, GEIGER, Christopher D. e LAMONT, Gary B. "FastPGA: A dynamic population sizing approach for solving expensive multiobjective optimization problems." International Conference on Evolutionary Multi-Criterion Optimization. Springer, Berlin, Heidelberg, 2007.

GAGNÉ, Christian e PARIZEAU, Marc. "Open BEAGLE: a C++ framework for your favorite evolutionary algorithm." ACM SIGEVolution 1.1 (2006): 12-15.

GREINER, David, et al. "Improving computational mechanics optimum design using helper objectives: An application in frame bar structures." International Conference on Evolutionary Multi-Criterion Optimization. Springer, Berlin, Heidelberg, 2007.

HALIM, Ronald Apriliyanto e Mamadou D. Seck. "The simulation-based multi-objective evolutionary optimization (SIMEON) framework." Proceedings of the 2011 Winter

Simulation Conference (WSC). IEEE, 2011.

DEB, K. e JAIN, H. "An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I: Solving Problems With Box Constraints," in IEEE Transactions on Evolutionary Computation, vol. 18, no. 4, pp. 577-601, Aug. 2014, doi: 10.1109/TEVC.2013.2281535.

KNOWLES, Joshua D. e CORNE, David W. "Approximating the nondominated front using the Pareto archived evolution strategy." Evolutionary computation 8.2 (2000): 149-172.

KOLLAT, Joshua B. e REED, Patrick. "A framework for visually interactive decision-making and design using evolutionary multi-objective optimization (VIDEO)." Environmental Modelling & Software 22.12 (2007): 1691-1704.

KUKKONEN, Saku, and LAMPINEN, Jouni. "GDE3: The third evolution step of generalized differential evolution." 2005 IEEE congress on evolutionary computation. Vol. 1. IEEE, 2005.

LEON, Coromoto, MIRANDA, Gara, e SEGURA, Carlos. "METCO: a parallel plugin-based framework for multi-objective optimization." International Journal on Artificial Intelligence Tools 18.04 (2009): 569-588.

LIEFOOGHE, Arnaud et al. "ParadisEO-MOEO: A framework for evolutionary multi-objective optimization." International conference on evolutionary multi-criterion optimization. Springer, Berlin, Heidelberg, 2007.

NEBRO, Antonio J. et al. "AbYSS: Adapting scatter search to multiobjective optimization." IEEE Transactions on Evolutionary Computation 12.4 (2008): 439-457.

NEBRO, Antonio J. et al. "Design issues in a multiobjective cellular genetic algorithm." International Conference on Evolutionary Multi-Criterion Optimization. Springer, Berlin, Heidelberg, 2007.

NEBRO, Antonio J. et al. "Optimal antenna placement using a new multi-objective CHC algorithm." Proceedings of the 9th annual conference on Genetic and evolutionary

computation. 2007.

NEBRO, Antonio J., et al. "SMPSO: A new PSO-based metaheuristic for multi-objective optimization." 2009 IEEE Symposium on computational intelligence in multi-criteria decision-making (MCDM). IEEE, 2009.

ZHANG, Qingfu e LI, Hui. MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition. Em IEEE Transactions on Evolutionary Computation, v. 11, p. 712 - 731, 2007.

SIERRA, Margarita e COELLO, Carlos A. Coello. "Improving PSO-based multi-objective optimization using crowding, mutation and ϵ -dominance." International conference on evolutionary multi-criterion optimization. Springer, Berlin, Heidelberg, 2005.

SRINIVAS, Nidamarthi DEB, Kalyanmoy. "Multiobjective optimization using nondominated sorting in genetic algorithms." Evolutionary computation 2.3 (1994): 221-248.

VENTURA, Sebastián et al. "JCLEC: a Java framework for evolutionary computation." Soft computing 12.4 (2008): 381-392.

ZITZLER, Eckart e THIELE, Lothar. "An evolutionary algorithm for multiobjective optimization: The strength pareto approach." TIK-report 43 (1998).

ZITZLER, Eckart e KÜNZLI, Simon. "Indicator-based selection in multiobjective search." International conference on parallel problem solving from nature. Springer, Berlin, Heidelberg, 2004.

COELLO, Carlos e SIERRA, Margarita. "A study of the parallelization of a coevolutionary multi-objective evolutionary algorithm." MICAI 2004: Advances in Artificial Intelligence. p. 688 - 697.

DEB, Kalyanmoy et al. "A taxonomy for metamodeling frameworks for evolutionary multi-objective optimization." IEEE Transactions on Evolutionary Computing, v. 23, 2019.

DEB, Kalyanmoy, SINDHYA, Karthik e OKABE, Tatsuya. “Self-adaptive simulated binary crossover for real-parameter optimization.” Em Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO ‘07, p. 1187 - 1194, 2007. ACM.

ZITZLER, Eckart, DEB, Kalyanmoy e THIELE, Lothar. “Comparison of multiobjective evolutionary algorithms: empirical results.” Evolutionary Computation, 173–195, 2000.

DEB, K. “Multi-objective genetic algorithms: Problem difficulties and construction of test problems.” Evolutionary Computation, 7: 205–230, 1999.

OLIVEIRA, Joel. “Avaliação de técnicas de aprendizagem de máquina como surrogate na otimização com muitos objetivos.” Dissertação (Mestrado em Ciência da Computação), Universidade Federal de Sergipe, 2020.

BREIMAN, Leo. “Random Forests.” Machine Learning 45, 5-32 (2001).

JONES, D. R. “A taxonomy of global optimization methods based on response surfaces.” Journal of global optimization, vol. 21, no. 4, pp. 345–383, 2001.

8. Outras atividades

Foram realizadas reuniões semanais, a fim de tirar dúvidas, relatar o progresso e receber orientações. Além disso, foram feitos dois cursos de aprendizagem de máquina, com o propósito de facilitar a compreensão e a aplicação dos surrogates.

9. JUSTIFICATIVA DE ALTERAÇÃO NO PLANO DE TRABALHO

Não houve alterações no plano de trabalho, mas alguns objetivos definidos no plano de trabalho não foram alcançados. No decorrer do projeto, no mês de junho de 2021, houve troca de bolsistas neste plano, pois o aluno inicialmente alocado solicitou desligamento. O bolsista que estava alocado em outro plano de trabalho do mesmo projeto foi realocado neste plano para aproveitar uma bolsa COPES/PNAES. Assim, as atividades desenvolvidas no final da pesquisa se concentraram na escrita do artigo e na integralização dos códigos desenvolvidos nos dois planos.